

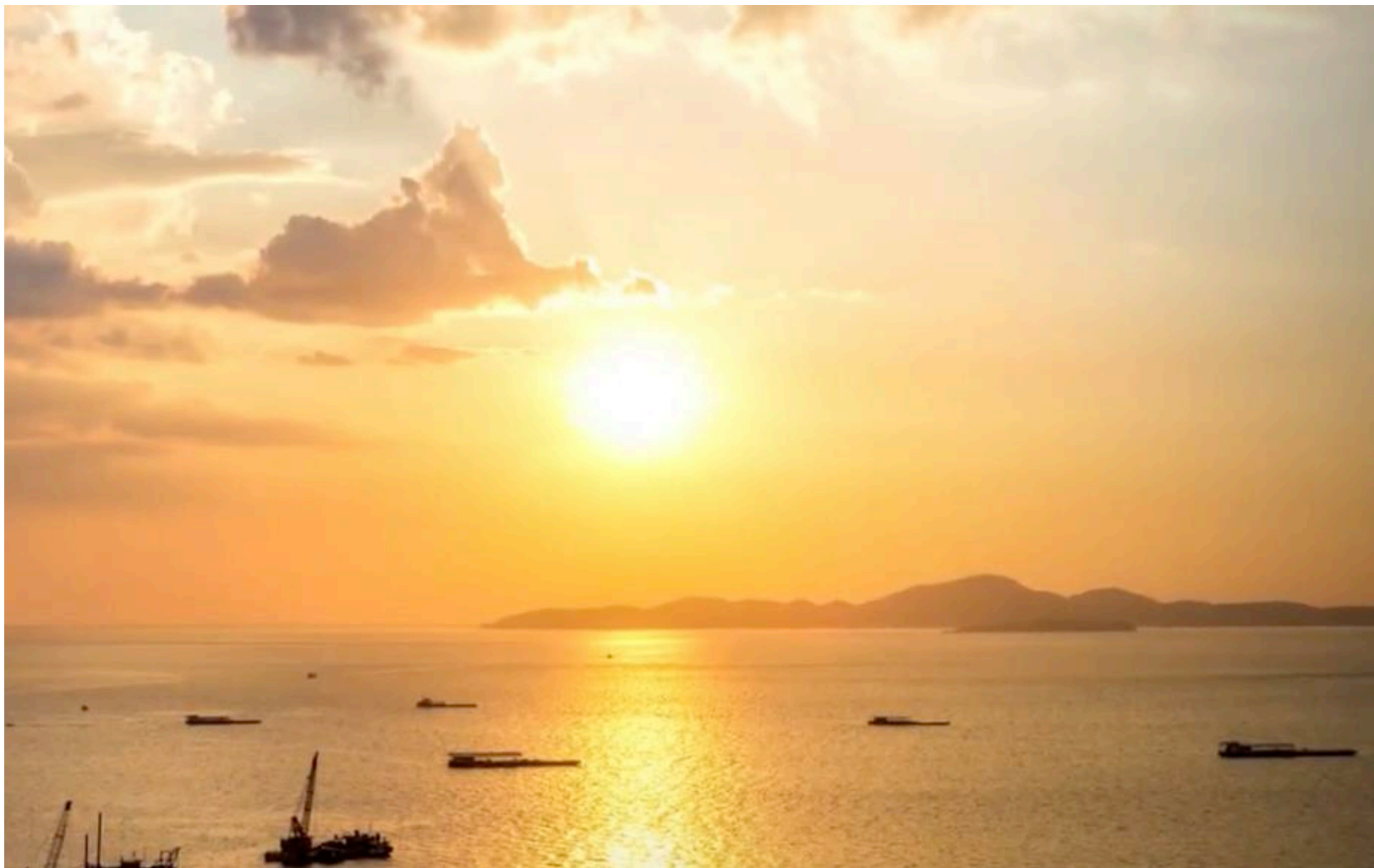


Pruebas

Programación con JavaScript II



Actividad de Bienestar





	Temas
1	Manejadores de paquetes y empaquetados de módulos
2	Nuevos tipos y características
3	Asincronía
4	API
5	Ajax
6	Scope y This
7	Transpiladores
8	Pruebas





Prework

- Instalar la extensión JEST en Visual Studio Code.
- Leer detenidamente y comprender el material explicado en el tema **Pruebas**.
- Practicar todos los ejemplos que se describen previamente en el tema **Pruebas**.
- Revisar cada uno de los recursos adicionales que se proponen en el tema **Pruebas**.



Para saber más

Lecturas

- Facebook Open Source. (2022). Documentación JEST. Recuperado de <https://devdocs.io/jest/>
- Martinez, M. (2021). Introducción al unit testing en JavaScript con JEST. Recuperado de <https://40q.agency/introduccion-al-unit-testing-en-javascript-con-jest/>



Para saber más

Videos

- Raúl Palacios. (2021-07-18). 01 JEST Introducción [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=IP Ae1N0GsFM>
- Dev Done. (2022-03-11). Jest VSCode Extension | How to code Tutorial [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=zWYmDO5UzWQ>



Sesión Guiada 8

Objetivo: Introducir y consolidar el uso de pruebas unitarias en JavaScript mediante Jest, desarrollando habilidades para escribir, ejecutar y analizar pruebas que aseguren la calidad y funcionamiento correcto del código.

Aspectos clave de la sesión:

- Comprender los fundamentos y tipos de pruebas automatizadas.
- Aprender a configurar y utilizar Jest para pruebas unitarias.
- Aplicar diferentes matchers para validar distintos tipos de datos y casos.
- Practicar con ejercicios que refuercen el pensamiento crítico y la detección temprana de errores.



Actividad Guiada

Parte 1

Duración: 75 minutos.



Ejercicio 1. Objetivo

Comprender y **aplicar el uso de Jest** para realizar pruebas básicas sobre operaciones aritméticas y validación de datos mediante matchers.

- Verificar el comportamiento de una operación simple (suma) usando pruebas unitarias.
- Aplicar matchers como `.toBe()`, `.toBeGreaterThan()`, `.toBeLessThanOrEqual()`, etc.
- Realizar pruebas con strings utilizando `.toMatch()` y `.not.toMatch()`.
- Generar y analizar un **reporte de cobertura** que refleje qué parte del código fue ejecutada durante las pruebas.



¿Qué tipo de pruebas se pueden realizar a un código?



Pruebas unitarias (unit tests):

Verifican que **funciones individuales** del código hagan lo que se espera.



Pruebas de integración (integration tests):

Evalúan cómo **interactúan varias partes** del sistema entre sí.



Pruebas de interfaz (end-to-end / E2E):

Simulan el comportamiento del usuario en la aplicación completa.



Explica qué es JEST y por qué nos interesaría utilizarla.

Jest es una herramienta (framework) para realizar **pruebas automatizadas** en JavaScript. Fue desarrollada por **Meta (antes Facebook)** y es ampliamente utilizada en proyectos modernos.

Ventajas de usar Jest:



Es fácil de configurar.



Es rápido (usa ejecución paralela).



Tiene funcionalidades integradas como mocks, cobertura de código y matchers.



No necesita muchas dependencias externas.



Explica qué es JEST y por qué nos interesaría utilizarla.

¿Por qué usar Jest?

Porque nos ayuda a **detectar errores** de forma anticipada, a asegurar que nuestros cambios no rompan el sistema y a tener **confianza** en el código que escribimos.



¿Qué es un **matcher**?

En Jest, un **matcher** es una función que se utiliza para **comparar valores** esperados y reales durante una prueba.

```
expect(2 + 2).toBe(4);
```

Es un **matcher** que verifica que el resultado sea exactamente 4.

Otros matchers comunes:

`.toEqual(obj)` – compara objetos o arrays.

`.toBeTruthy()` – verifica si algo es verdadero.

`.toBeNull()` – verifica si es null.

`.toContain(item)` – verifica si un array contiene un elemento.

```
npm init -y  
npm install --save-dev jest
```

Revisa el siguiente código

ejercicio1.js

```
describe('Ejercicio 1', () => {  
  test('Prueba', () => {  
    const value = 2+2;  
    expect(value).toBeGreaterThan(3);  
    expect(value).toBeGreaterThanOrEqual(4);  
    expect(value).toBeLessThan(5);  
    expect(value).toBeLessThanOrEqual(4);  
  
    //toBe y toEqual son equivalentes al usar números  
    expect(value).toBe(4);  
    expect(value).toEqual(4);  
  });  
});
```

Completa los espacios del código para que cada una de las pruebas sea exitosa.

```
npm init -y  
npm install --save-dev jest
```



```
{  
  "name": "ejercicios",  
  "version": "1.0.0",  
  "main": "ejercicio1.js",  
  "scripts": {  
    "test": "jest --coverage"  
  },  
  "author": "",  
  "license": "ISC",  
  "description": "",  
  "devDependencies": {  
    "jest": "^29.7.0"  
  }  
}
```



Pruebas con String

Incluye un par de pruebas en el código anterior que permitan realizar evaluaciones del código string con los matchers:

- `not.toMatch(string)`
- `toMatch(string)`

Genera el reporte de cobertura desde la terminal y desde el archivo `index.html`.



Pruebas con String

```
describe('Pruebas con strings', () => {  
  test('Comparación de texto', () => {  
    const mensaje = 'Hola mundo';  
  
    expect(mensaje).toMatch(/mundo/); // contiene la palabra "mundo"  
    expect(mensaje).not.toMatch(/adiós/); // no contiene la palabra "adiós"  
  });  
});
```



Reporte de cobertura

```
"scripts": {  
  "test": "jest --coverage"  
},
```

package.json

```
% npm install --save-dev jest
```

Terminal



Reporte de cobertura

```
leticiapalossanchez@macmini Ejercicios % npm test
```

```
> ejercicios@1.0.0 test
```

```
> jest --coverage
```

```
PASS ./ejercicio1.test.js
```

```
Ejercicio 1
```

```
✓ Prueba con número (2 ms)
```

```
Pruebas con strings
```

```
✓ Comparación de texto
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
sum.js	100	100	100	100	

```
Test Suites: 1 passed, 1 total
```

```
Tests: 2 passed, 2 total
```

```
Snapshots: 0 total
```

```
Time: 0.224 s, estimated 1 s
```

```
Ran all test suites.
```



Columna	Significado	Ejemplo práctico
File	El archivo de código fuente que fue evaluado.	sum.js
% Stmts	Porcentaje de sentencias (statements) ejecutadas durante las pruebas.	return, let, etc.
% Branch	Porcentaje de ramas condicionales cubiertas (if, else, ternarios, switch).	if (a > b)
% Funcs	Porcentaje de funciones ejecutadas por las pruebas. Incluye funciones normales y flecha.	function sumar()
% Lines	Porcentaje de líneas de código que fueron ejecutadas. Es similar a % Stmts.	Línea 5 del archivo
Uncovered Line #s	Muestra los números de línea que no fueron cubiertos por las pruebas.	15, 18-20



Ejercicio 2. Objetivo

Practicar la creación de pruebas unitarias en Jest para funciones con diferentes tipos de datos y evaluar el manejo de valores esperados y no esperados.

- Asegurar que una función retorne valores correctos según los parámetros recibidos.
- Utilizar diferentes matchers para validar resultados, incluyendo manejo de valores inexistentes o vacíos.
- Fomentar la comprensión de cómo las pruebas ayudan a garantizar el correcto funcionamiento de funciones en distintos escenarios.



Ejercicio 2:

1. Crea cuatro funciones en un archivo llamado **operaciones.js** que realicen las funciones básicas de sumar, restar, multiplicar y dividir.

```
const sumar = (a, b) => a + b;  
const restar = (a, b) =>         ;  
const multiplicar = (a, b) =>         ;  
const dividir = (a, b) =>         ;  
  
module.exports = {  
  sumar,  
  restar,  
  multiplicar,  
  dividir  
};
```



Ejercicio 2:

2. Crea un archivo llamado **operaciones.test.js** con el que realizarás las pruebas del código, asegurándote de que se encuentre en la misma carpeta.

Copia el siguiente código y completa las líneas indicadas para realizar las pruebas necesarias.



index.test.js

```
//Importamos las funciones de index.js
var {sumar,restar,multiplicar,dividir} = require('./index.js');

//Con Describe describimos un conjunto de pruebas a realizar
describe('Operaciones matemáticas', () => {
  //Con test definimos una prueba específica
  test('Suma de dos números', () => {
    const resultado = sumar(2, 3);
    expect(resultado).toBe(5); // Verifica que el resultado sea 5
  });

  test('Resta de dos números', () => {
    // [Redacted code block]
  });

  test('Multiplicación de dos números', () => {
    // [Redacted code block]
  });

  test('División de dos números', () => {
    // [Redacted code block]
  });
});
```



3. Ejecuta el reporte npm test desde la terminal de Visual Studio Code.

Haz los ajustes necesarios tanto al archivo `operaciones.js` como al archivo `operaciones.test.js` para asegurarte de que la prueba sea exitosa.



Ejercicio 3. Objetivo

Desarrollar habilidades para escribir pruebas unitarias que validen comportamientos condicionales y la correcta respuesta de funciones ante distintos inputs.

- Evaluar cómo una función responde a diferentes valores de entrada.
- Aplicar matchers para verificar condiciones positivas y negativas.
- Comprender la importancia de probar casos límite y valores inesperados.
- Profundizar en la lógica de las pruebas para mejorar la calidad del código.



Ejercicio 3: Analiza el código

array.js

```
const estados = ['Aguascalientes', 'Baja California', 'Baja California Sur', 'Campeche', 'Chiapas', 'Chihuahua',  
                'Coahuila', 'Colima', 'Ciudad de México', 'Durango', 'Guanajuato', 'Guerrero', 'Hidalgo', 'Jalisco',  
                'México', 'Michoacán', 'Morelos', 'Nayarit', 'Nuevo León', 'Oaxaca', 'Puebla', 'Querétaro', 'Quintana Roo',  
                'San Luis Potosí', 'Sinaloa', 'Sonora', 'Tabasco', 'Tamaulipas', 'Tlaxcala', 'Veracruz', 'Yucatán', 'Zacatecas'];  
  
const dias = ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado', 'Domingo'];  
  
module.exports = {  
  estados,  
  dias  
};
```

array.test.js

```
// Importamos las funciones de array.js  
var { estados, dias } = require('./array.js');  
  
test('Revisamos el contenido de los arreglos', () => {  
  expect(estados).toContain('Aguascalientes');  
  expect(new Set(estados)).toContain('Aguascalientes');  
});
```



1. ¿Qué hace la prueba al arreglo estados?

```
expect(estados).toContain('Aguascalientes');
```

Esta línea **verifica que el string 'Aguascalientes' esté incluido en el arreglo estados.**

 Es una prueba común para asegurarse de que un elemento específico existe dentro del array.



2. ¿Qué pretende hacer la prueba con `new Set(estados)`?

```
expect(new Set(estados)).toContain('Aguascalientes');
```

Esta prueba **hace lo mismo que la anterior, pero con una diferencia clave**: antes de hacer la verificación, convierte el array en un Set.

¿Qué es un Set?

Un **Set** en JavaScript es una estructura de datos que:



Elimina duplicados automáticamente



Solo contiene valores únicos



Receso

Duración: 10 minutos.



Actividad Guiada

Parte 2

Duración: 75 minutos.



Ejercicio 4. Objetivo

Fortalecer la capacidad de crear pruebas unitarias en Jest para validar la existencia y comportamiento esperado de funciones que manipulan datos y retornan resultados específicos.

- Verificar que las funciones a probar existan y sean del tipo esperado.
- Comprobar que las funciones retornen valores correctos según diferentes casos de uso.
- Manejar entradas válidas, inválidas, vacías o nulas para garantizar robustez.
- Familiarizarse con el ciclo completo de escribir, ejecutar y evaluar pruebas.



Ejercicio 4

1. Analiza el siguiente código y explica cuál es el objetivo de la prueba

```
Ejercicios > JS lenguajes.js > ...
1  const idiomaIngles = 'en-US';
2  const idiomaEspañol = 'es-ES';
3
4  function lenguajePagina(language){
5      switch (language.toLowerCase()) {
6          case idiomaIngles.toLowerCase():
7              return '/about-us';
8          case idiomaEspañol.toLowerCase():
9              return '/acerca-de-nosotros';
10     }
11     return '';
12 }
13
14 module.exports = lenguajePagina
```

```
const lenguajePagina = require('./lenguajes');

test('Revisamos el valor de retorno del lenguaje seleccionado', () => {
    expect(lenguajePagina('es-ES')).toBe('/about-us');
});
```



Genera el reporte de cobertura e identifica el porcentaje de aprobación.

```
PASS ./index.test.js
PASS ./ejercicio1.test.js
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	90.47	50	100	88.23	
array.js	100	100	100	100	
index.js	100	100	100	100	
lenguajes.js	71.42	50	100	71.42	7,11
sum.js	100	100	100	100	

```
Test Suites: 1 failed, 3 passed, 4 total
Tests:       1 failed, 7 passed, 8 total
Snapshots:   0 total
Time:        0.244 s, estimated 1 s
Ran all test suites.
```



¿A qué se debe que la prueba falle?

FAIL ./lenguajes.test.js

● Revisamos el valor de retorno del lenguaje seleccionado

expect(received).toBe(expected) // Object.is equality

Expected: `"/about-us"`

Received: `"/acerca-de-nosotros"`

```
2 |  
3 | test('Revisamos el valor de retorno del lenguaje seleccionado', () => {  
> 4 |     expect(lenguajePagina('es-ES')).toBe('/about-us');  
   |                                     ^  
5 | });  
6 |  
7 |
```

at Object.toBe (lenguajes.test.js:4:37)

PASS ./array.test.js

PASS ./index.test.js

PASS ./ejercicio1.test.js



¿Qué líneas de código cambiarías para que la prueba funcione?

```
test('Revisamos el valor de retorno del lenguaje selecionado', () => {  
  expect(lenguajePagina('en-US')).toBe('/about-us');  
  expect(lenguajePagina('es-ES')).toBe('/acerca-de-nosotros');  
});
```

Si se agregara el idioma francés con el valor “fr-FR” como un tercer lenguaje en una página web, ¿qué líneas agregarías y cuál sería la prueba?

```
const idiomaFrances = 'fr-FR';
```

```
case idiomaFrances.toLowerCase():  
  return '/a-propos';
```

```
expect(lenguajePagina('fr-FR')).toBe('/a-propos');
```



Genera el reporte de cobertura e identifica el porcentaje de las funciones que pasaron la prueba.

```

leticiapalossanchez@macmini Ejercicios % npm test

> ejercicios@1.0.0 test
> jest --coverage

PASS ./lenguajes.test.js
PASS ./array.test.js
PASS ./index.test.js
PASS ./ejercicio1.test.js

```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	95.65	100	100	94.73	
array.js	100	100	100	100	
index.js	100	100	100	100	
lenguajes.js	88.88	100	100	88.88	14
sum.js	100	100	100	100	

```

Test Suites: 4 passed, 4 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        0.266 s, estimated 1 s
Ran all test suites.

```



Ejercicio 5. Objetivo

Diseñar y ejecutar pruebas unitarias que validen el comportamiento de una función que retorna descripciones basadas en un objeto mapeado, asegurando la correcta gestión de parámetros válidos, inválidos, vacíos o ausentes.

- Confirmar que la función getObjectDescription existe y es invocable.
- Verificar la respuesta correcta cuando se pasa un valor existente en el objeto mapeo.
- Evaluar el manejo de parámetros que no están definidos en el objeto.
- Probar la función con parámetros vacíos o sin parámetros para validar mensajes de error o respuestas controladas.
- Reforzar la importancia de cubrir distintos escenarios para mejorar la confiabilidad del código.



Ejercicio 5

Crea el archivo objectMapping.js

```
const objectMapping = {  
  ACTIVO: 'Activo',  
  INACTIVO: 'Inactivo',  
  OBSOLETO: 'Obsoleto',  
};  
  
function getObjectDescription(tipo) {  
  if(!tipo) {  
    return "El parámetro 'tipo' no existe";  
  }  
  return objectMapping[tipo];  
}  
  
module.exports = getObjectDescription;
```



Crea un archivo `objectMapping.test.js` con las siguientes pruebas:

- a. Garantiza que `getObjectDescription` exista.
- b. Usa el método `getObjectDescription` con un parámetro que existe en el objeto `objectMapping`.
- c. Usa el método `getObjectDescription` con un parámetro que no exista en el `objectMapping`.
- d. Usa el método `getObjectDescription` con un parámetro vacío.
- e. Usa el método `getObjectDescription` sin parámetro.



objectMapping.test.js

```
const getObjectDescription = require('./objectMapping');

describe('Pruebas para getObjectDescription', () => {

  // a. Garantiza que "getObjectDescription" exista.
  test('a. La función getObjectDescription debe existir', () => {
    expect(typeof getObjectDescription).toBe('function');
  });

  // b. Usa el método con un parámetro que existe en el objeto.
  test('b. Retorna "Activo" cuando el parámetro es "ACTIVO"', () => {
    expect(getObjectDescription('ACTIVO')).toBe('Activo');
  });

  // c. Usa el método con un parámetro que no exista.
  test('c. Retorna undefined cuando el parámetro no está en el objeto', () => {
    expect(getObjectDescription('ARCHIVADO')).toBeUndefined();
  });

  // d. Usa el método con un parámetro vacío.
  test('d. Retorna mensaje cuando el parámetro está vacío', () => {
    expect(getObjectDescription('')).toBe("El parámetro 'tipo' no existe");
  });

  // e. Usa el método sin parámetro.
  test('e. Retorna mensaje cuando no se pasa ningún parámetro', () => {
    expect(getObjectDescription()).toBe("El parámetro 'tipo' no existe");
  });

});
```



Ejercicio 6. Objetivo

Implementar pruebas unitarias para validar funciones que realizan operaciones numéricas básicas, tales como suma, filtrado de números positivos y negativos, asegurando que los resultados sean correctos y manejando distintos escenarios de entrada.

- Verificar que la función suma calcula correctamente la suma de un arreglo de números.
- Comprobar que las funciones positivos y negativos filtran adecuadamente los valores según su signo.
- Evaluar el comportamiento de las funciones ante entradas variadas, incluyendo arreglos vacíos y combinaciones de valores.



Ejercicio 6

Crea el archivo

valoresNumericos.js

```
const suma = (vals) => {  
  let suma = 0;  
  vals.forEach((val) => {  
    suma += val;  
  });  
  return suma;  
};  
  
const positivos = (valores) => {  
  return valores.filter((x) => x > 0);  
};  
  
const negativos = (valores) => {  
  return valores.filter((x) => x < 0);  
};  
  
module.exports = {  
  suma,  
  positivos,  
  negativos  
};
```



Escribe las pruebas que realizarías para saber si el código devuelve los valores correctos. Escribe el código JEST en un archivo llamado valoresnumericos.test.js y evalúa las pruebas que diseñaste en el paso anterior.

```
const { suma, positivos, negativos } = require('./valoresNumericos');  
describe('Pruebas para la función suma', () => {  
  test('Suma de números positivos', () => {  
    expect(suma([1, 2, 3])).toBe(6);  
  });  
  
  test('Suma con números negativos y positivos', () => {  
    expect(suma([-1, 2, -3, 4])).toBe(2);  
  });  
  
  test('Suma de arreglo vacío', () => {  
    expect(suma([])).toBe(0);  
  });  
});
```



valoresNumericos.test.js Part II

```
describe('Pruebas para la función positivos', () => {  
  test('Filtra solo números positivos', () => {  
    expect(positivos([-2, 0, 3, 5, -1])).toEqual([3, 5]);  
  });  
  
  test('Sin positivos', () => {  
    expect(positivos([-1, -5, -10])).toEqual([]);  
  });  
  
  test('Todos son positivos', () => {  
    expect(positivos([1, 2, 3])).toEqual([1, 2, 3]);  
  });  
});
```

```
describe('Pruebas para la función negativos', () => {  
  test('Filtra solo números negativos', () => {  
    expect(negativos([-2, 0, 3, -1])).toEqual([-2, -1]);  
  });  
  
  test('Sin negativos', () => {  
    expect(negativos([0, 2, 4])).toEqual([]);  
  });  
  
  test('Todos son negativos', () => {  
    expect(negativos([-3, -4, -5])).toEqual([-3, -4, -5]);  
  });  
});
```



¿Qué cambiarías en el código original para saber si son números primos?

```
const esPrimo = (n) => {  
  if (n <= 1 || !Number.isInteger(n)) return false;  
  for (let i = 2; i <= Math.sqrt(n); i++) {  
    if (n % i === 0) return false;  
  }  
  return true;  
};  
  
const primos = (valores) => {  
  return valores.filter(esPrimo);  
};  
  
module.exports = {  
  suma,  
  positivos,  
  negativos,  
  primos  
};
```

valoresNumericos.js



valoresNumericos.test.js

```
const { suma, positivos, negativos, primos } = require('./valoresNumericos');
```

```
describe('Pruebas para la función primos', () => {  
  test('Filtra los números primos en un arreglo', () => {  
    expect(primos([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])).toEqual([2, 3, 5, 7]);  
  });  
  
  test('Ningún número primo', () => {  
    expect(primos([0, 1, -3, 4, 6, 8, 9])).toEqual([]);  
  });  
  
  test('Todos son primos', () => {  
    expect(primos([2, 3, 5, 7, 11])).toEqual([2, 3, 5, 7, 11]);  
  });  
  
  test('Incluye valores no enteros o negativos', () => {  
    expect(primos([2, 3.5, -7, 7])).toEqual([2, 7]); // solo enteros positivos  
  });  
});
```



Realiza las pruebas necesarias para garantizar si son correctos.

Demuestra que tus pruebas obtienen un 100% en el reporte de cobertura de JEST.

```
leticiapalossanchez@macmini Ejercicios % npm test
```

```
> ejercicios@1.0.0 test
> jest --coverage
```

```
PASS ./valoresNumericos.test.js
PASS ./objectMapping.test.js
PASS ./array.test.js
PASS ./lenguajes.test.js
PASS ./index.test.js
PASS ./ejercicio1.test.js
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	98	100	100	97.56	
array.js	100	100	100	100	
index.js	100	100	100	100	
lenguajes.js	88.88	100	100	88.88	14
objectMapping.js	100	100	100	100	
sum.js	100	100	100	100	
valoresNumericos.js	100	100	100	100	

```
Test Suites: 6 passed, 6 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        0.325 s, estimated 1 s
Ran all test suites.
```



Cierre

Duración: 10 minutos.



Importancia de las pruebas Unitarias

- Garantizan la calidad y confiabilidad del código
- Detectan errores de forma temprana
- Facilitan el mantenimiento a largo plazo



Invitación a la práctica

- Aplica lo aprendido en tus proyectos
- Explora herramientas y técnicas de testing
- Practica constantemente para mejorar



Encuesta



ID OPINA

SCACL004860



¡Recuerda!

Un código bien probado es un código más seguro, eficiente y confiable.



¡Gracias!